

Software Design Patterns and Principles

Assignment

Java Implementation Submissions • Verified Solutions Reference Document

Exercise 1: Implementing the Singleton Pattern

SCENARIO

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

ANSWER IMPLEMENTATION

```
package SingletonPatternExample;

/**
 * Singleton implementation of a Logger utility class.
 * Ensures a single instantiation point and prevents external direct allocation.
 */
public class Logger {
    // Private static reference to hold the single unique instance of this class
    private static Logger instance;

    // Private constructor blocks access from the 'new' keyword externally
    private Logger() {
        System.out.println("Logger Instance Initialized.");
    }

    // Thread-safe global access point utilizing lazy initialization
    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    // Business method exposed by the singleton
    public void log(String message) {
        System.out.println("[LOG]: " + message);
    }
}
```

Exercise 2: Implementing the Factory Method Pattern

SCENARIO

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

ANSWER IMPLEMENTATION

```
package FactoryMethodPatternExample;

// Abstract product interface
interface Document {
    void open();
}

// Concrete products
class WordDocument implements Document {
    public void open() { System.out.println("Opening Word Document."); }
}
class PdfDocument implements Document {
    public void open() { System.out.println("Opening PDF Document."); }
}
class ExcelDocument implements Document {
    public void open() { System.out.println("Opening Excel Document."); }
}

// Abstract Creator class defining the Factory Method
abstract class DocumentFactory {
    public abstract Document createDocument();
}

// Concrete Creators overriding the Factory Method
class WordFactory extends DocumentFactory {
    public Document createDocument() { return new WordDocument(); }
}
class PdfFactory extends DocumentFactory {
    public Document createDocument() { return new PdfDocument(); }
}
class ExcelFactory extends DocumentFactory {
    public Document createDocument() { return new ExcelDocument(); }
}
```

Exercise 3: Implementing the Builder Pattern

SCENARIO

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

ANSWER IMPLEMENTATION

```
package BuilderPatternExample;

/**
 * Complex product built step-by-step using an internal Builder.
 */
public class Computer {
    private final String CPU;
    private final String RAM;
    private final String Storage;

    // Private constructor accepting the nested Static Builder reference
    private Computer(Builder builder) {
        this.CPU = builder.CPU;
        this.RAM = builder.RAM;
        this.Storage = builder.Storage;
    }

    // Static nested Builder class
    public static class Builder {
        private String CPU;
        private String RAM;
        private String Storage;

        public Builder setCPU(String CPU) { this.CPU = CPU; return this; }
        public Builder setRAM(String RAM) { this.RAM = RAM; return this; }
        public Builder setStorage(String Storage) { this.Storage = Storage; return this; }

        // Build method instantiating the immutable Computer object
        public Computer build() { return new Computer(this); }
    }

    @Override
    public String toString() {
        return "Computer Configuration: [CPU=" + CPU + ", RAM=" + RAM + ", Storage=" +
Storage + "]\n";
    }
}
```

Exercise 4: Implementing the Adapter Pattern

SCENARIO

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

ANSWER IMPLEMENTATION

```
package AdapterPatternExample;

// Expected internal system Target interface
interface PaymentProcessor {
    void processPayment(double amount);
}

// Incompatible external Adaptee class
class PayPalGateway {
    public void makePayPalPayment(double dollars) {
        System.out.println("Paid $" + dollars + " via PayPal API interface.");
    }
}

// Structural Adapter translating Target interface calls into Adaptee executions
class PayPalAdapter implements PaymentProcessor {
    private PayPalGateway payPalGateway;

    public PayPalAdapter(PayPalGateway payPalGateway) {
        this.payPalGateway = payPalGateway;
    }

    @Override
    public void processPayment(double amount) {
        payPalGateway.makePayPalPayment(amount);
    }
}
```

Exercise 5: Implementing the Decorator Pattern

SCENARIO

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

ANSWER IMPLEMENTATION

```
package DecoratorPatternExample;

// Component Interface
interface Notifier {
    void send(String message);
}

// Concrete Component providing baseline delivery functionality
class EmailNotifier implements Notifier {
    public void send(String message) { System.out.println("Sending Email: " + message); }
}

// Abstract Decorator holding structural composition reference
```

```

abstract class NotifierDecorator implements Notifier {
    protected Notifier wrappedNotifier;

    public NotifierDecorator(Notifier notifier) { this.wrappedNotifier = notifier; }
    public void send(String message) { wrappedNotifier.send(message); }
}

// Concrete wrapper adding SMS extension features dynamically
class SMSNotifierDecorator extends NotifierDecorator {
    public SMSNotifierDecorator(Notifier notifier) { super(notifier); }

    @Override
    public void send(String message) {
        super.send(message);
        System.out.println("Extending Notification -> Sending SMS Text: " + message);
    }
}

```

Exercise 6: Implementing the Proxy Pattern

SCENARIO

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

ANSWER IMPLEMENTATION

```

package ProxyPatternExample;

// Subject interface
interface Image {
    void display();
}

// Real Subject executing heavy instantiation behaviors
class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromRemoteServer();
    }
    private void loadFromRemoteServer() {
        System.out.println("Downloading high-resolution resource file from server: " +
filename);
    }
    public void display() { System.out.println("Rendering Image Canvas: " + filename); }
}

// Proxy implementing lazy initialization controls and reference caching
class ProxyImage implements Image {
    private String filename;

```

```

private RealImage realImage;

public ProxyImage(String filename) { this.filename = filename; }

public void display() {
    if (realImage == null) {
        System.out.println("[Cache Miss] Instantiating concrete connection
resource.");
        realImage = new RealImage(filename); // Lazy initialization loop
    } else {
        System.out.println("[Cache Hit] Reusing instantiated connection reference.");
    }
    realImage.display();
}
}

```

Exercise 7: Implementing the Observer Pattern

SCENARIO

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

ANSWER IMPLEMENTATION

```

package ObserverPatternExample;

import java.util.ArrayList;
import java.util.List;

// Observer interface listener signature
interface Observer {
    void update(String stockName, double price);
}

// Subject interface
interface Stock {
    void register(Observer o);
    void deregister(Observer o);
    void notifyObservers();
}

// Concrete subject updating data state and triggering client dispatches
class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockName;
    private double price;

    public void setStockPrice(String stockName, double price) {
        this.stockName = stockName;
        this.price = price;
        notifyObservers(); // Automatic state sync
    }
}

```

```

    }

    public void register(Observer o) { observers.add(o); }
    public void deregister(Observer o) { observers.remove(o); }
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(stockName, price);
        }
    }
}

// Concrete observer application endpoint
class MobileApp implements Observer {
    public void update(String stockName, double price) {
        System.out.println("Mobile Device Alert -> " + stockName + " valuation updated to: $" + price);
    }
}

```

Exercise 8: Implementing the Strategy Pattern

SCENARIO

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

ANSWER IMPLEMENTATION

```

package StrategyPatternExample;

// Abstract Strategy encapsulation rule
interface PaymentStrategy {
    void pay(double amount);
}

// Concrete algorithmic variants
class CreditCardPayment implements PaymentStrategy {
    private String cardNum;
    public CreditCardPayment(String cardNum) { this.cardNum = cardNum; }
    public void pay(double amount) { System.out.println("Paid $" + amount + " processing via Credit Card configuration."); }
}

class PayPalPayment implements PaymentStrategy {
    private String email;
    public PayPalPayment(String email) { this.email = email; }
    public void pay(double amount) { System.out.println("Paid $" + amount + " processing via PayPal Wallet configuration."); }
}

// Context configuration managing dynamically swapped behaviors
class PaymentContext {

```

```

private PaymentStrategy strategy;

public void setPaymentStrategy(PaymentStrategy strategy) { this.strategy = strategy; }
public void executePayment(double amount) {
    if (strategy == null) {
        System.out.println("Error: Strategy missing.");
        return;
    }
    strategy.pay(amount);
}
}

```

Exercise 9: Implementing the Command Pattern

SCENARIO

You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

ANSWER IMPLEMENTATION

```

package CommandPatternExample;

// Universal Command interface declaration
interface Command {
    void execute();
}

// Receiver component containing actual implementation business logic actions
class Light {
    public void turnOn() { System.out.println("Hardware Execution: Light circuit closed [ON]."); }
    public void turnOff() { System.out.println("Hardware Execution: Light circuit broken [OFF]."); }
}

// Concrete Command layers mapping functional requests to the Receiver logic
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.turnOn(); }
}

class LightOffCommand implements Command {
    private Light light;
    public LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.turnOff(); }
}

// Invoker structure triggering object commands without knowing their explicit execution routes
class RemoteControl {

```



```

private Command command;
public void setCommand(Command command) { this.command = command; }
public void pressButton() { command.execute(); }
}

```

Exercise 10: Implementing the MVC Pattern

SCENARIO

You are developing a simple web application for managing student records using the MVC pattern.

ANSWER IMPLEMENTATION

```

package MVCPatternExample;

// 1. The Model layer managing clean data attributes and entities
class Student {
    private String name;
    private String id;
    private String grade;

    public Student(String id, String name, String grade) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }
    // Accessor and Mutator scopes
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}

// 2. The View layer handling purely terminal UI presentation layouts
class StudentView {
    public void displayStudentDetails(String studentName, String studentId, String
studentGrade) {
        System.out.println("--- Student Profile Record ---");
        System.out.println("Identification ID: " + studentId);
        System.out.println("Full Name Name      : " + studentName);
        System.out.println("Assigned Grade   : " + studentGrade);
    }
}

// 3. The Controller mediating information mapping between Model mutations and View
changes
class StudentController {
    private Student model;
    private StudentView view;
}

```

```

public StudentController(Student model, StudentView view) {
    this.model = model;
    this.view = view;
}
public void setStudentGrade(String grade) { model.setGrade(grade); }
public String getStudentGrade() { return model.getGrade(); }

public void updateView() {
    view.displayStudentDetails(model.getName(), model.getId(), model.getGrade());
}
}

```

Exercise 11: Implementing Dependency Injection

SCENARIO

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

ANSWER IMPLEMENTATION

```

package DependencyInjectionExample;

// Repository Abstraction Interface layer
interface CustomerRepository {
    String findCustomerById(int id);
}

// Concrete Persistent storage interaction layer implementation
class CustomerRepositoryImpl implements CustomerRepository {
    public String findCustomerById(int id) {
        return "Customer object mapping entry found for registration lookup ID: " + id;
    }
}

// Application Service dependent completely on structural abstraction interfaces
class CustomerService {
    private final CustomerRepository customerRepository;

    // Decoupling via Constructor Dependency Injection pattern assignment
    public CustomerService(CustomerRepository repository) {
        this.customerRepository = repository;
    }

    public void showCustomer(int id) {
        String dataFetch = customerRepository.findCustomerById(id);
        System.out.println("Service layer response payload: " + dataFetch);
    }
}

```